Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 28

Hash Tables

Open Addressing (Ctd.)

Open Addressing

- Open addressing:
  - Probe the hash table at a sequence of indices H_0(key), H_1(key), ...
- Probing without reordering: Uniform, linear, quadratic, double-hash
  - Hybrid: quadratic or double-hash probing combined with "parallel linear" probing
- Probing with reordering: Designed to "beat" Uniform Probing
  - Robin Hood hashing: Modified linear probing. Expected time for (successful or unsuccessful) search is $O(\frac{1}{1-\alpha})$, for insertion $O(\frac{1}{(1-\alpha)^2})$. Successful searches stay within an additive constant of expectation, and unsuccessful ones within O(log n) w.h.p.
  - Cuckoo hashing: O(1) worst-case search time. Insertion O(1) for small enough α.

Cuckoo Hashing

- We use d hash functions $H_1,\ldots,H_d$ where d is small (think of d=4)
- If key is stored, it will be at index $\in \{ H_1(key),\ldots,H_d(key) \}$: O(1) lookup time!
- Insertion algorithm, cuckoo_insert(item, $H_1(key),\ldots,H_d(key)$):
  - Look for an empty slot in the given set. If found, insert item there.
  - If no empty slot:
    - insert into one of the given slots (at random), "evicting" the item already there
    - recursively, cuckoo_insert the evicted item into one of its other d-1 slots
- Key question: (how fast) will cuckoo_insert terminate?

Cuckoo Hashing

- Only with α^d probability do we need to start on the eviction path
  - E.g., at α=0.8, d=4, about 0.4 probability
- Once we do start on the eviction path:
  - For a given d, there is a critical load factor α_{max}
    - For d=4, $\alpha_{max} \approx 0.97$. For d=2, $\alpha_{max} \approx 0.5$.
  - If $\alpha < \alpha_{max}$ probability of needing an additional step $\approx \alpha_{effective} := \frac{\alpha}{\alpha_{max}}$
    - Expected number of steps $\approx O(\frac{1}{1 - \alpha_{effective}})$
  - If $\alpha \geq \alpha_{max}$, almost surely goes into an infinite loop!

Cuckoo Hashing

- α_{max} is a "phase-boundary"
  - For $\alpha < \alpha_{max}$ we terminate fairly quickly
    - If $\alpha < \alpha_{max} - \varepsilon$ (where $\varepsilon > 0$ a constant independent of n) then expected number of steps during insertion = $O(1/\varepsilon) = O(1)$, and highly unlikely we go, say, 5x above the expectation
  - For $\alpha \geq \alpha_{max}$, if we start on the eviction path, we almost surely go into an infinite loop!
- If insertion takes too many steps, then we we're very likely to be in the infinite loop phase (assuming a cryptographically strong hash)
  - Fix: Reduce α. i.e., grow the hash table (double it) and rehash
- Alternately, before α gets too close to α_{max}, grow and rehash (and use a cryptographically strong hash)

Attendance Break!

Login using CSE LDAP id at

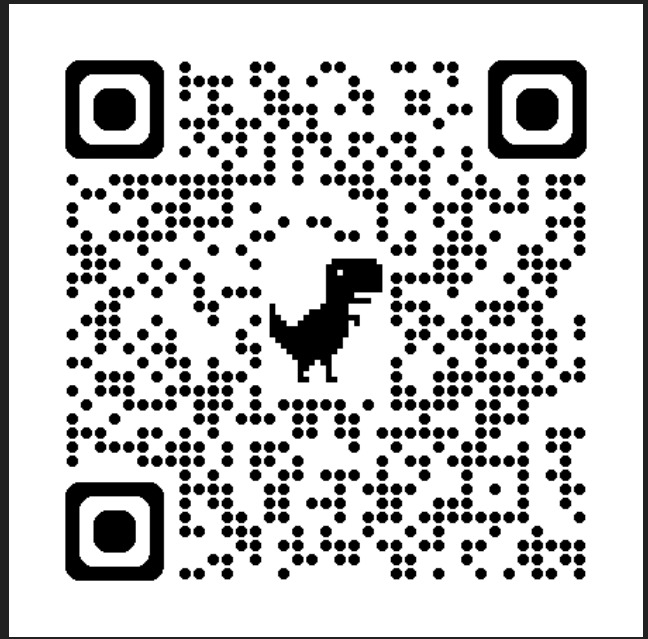
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Cuckoo Hashing with Parallel Checks

- Recall "parallel linear" probing:
  - Hash function yields an index to a "bucket" of B items
  - Fingerprints of all B items are checked in parallel (supported by hardware) against the current key's fingerprint. Full key checked only when fingerprint matches
- Extends to cuckoo hashing too: d hashes yield indices for d B-sized buckets. All Bd slots can be checked in one cycle.
- By increasing B, we can decrease d, without affecting α_{max}
  - E.g., d=2, B=4 has almost the same effect as d=4, B=1
  - Lower d is better (fewer hash functions to evaluate, fewer random locations to fetch), and a slightly large B is for free (still only one cycle)

Handling Adversarial Inputs

- Efficiency guarantees break down if the hash function doesn't behave like a uniformly random function (possibly under adversarial inputs)
- Can be prevented by using (light-weight) cryptographic hash functions
  - Against non-adaptive adversaries, enough to use a combinatorial hash function
- Alternately, can use a fallback mechanism on encountering a long eviction path/ probe path when load factor is small
  - Migration: Migrate the hash table to an alternate data structure (e.g., balanced BST, or a closed-addressing hash table with BST for each bucket)
  - Spillover stash: Add the input in hand to a stash implemented using an alternate data structure. When searching, first search in hash table, and then in the stash.

Hash Table Summary

- Main feature: practically O(1) lookup
  - In cuckoo hashing, guaranteed worst-case O(1)
  - In other open addressing schemes, expected O(1) time, assuming uniform hashing and $\alpha < 1 - \varepsilon$
    - In Robin Hood hashing, stays very close to $O(1/\varepsilon)$ for successful searches w.h.p. (and O(log n) for unsuccessful searches)
  - In closed addressing, assuming uniform hashing and $\alpha = O(1)$, all buckets remain practically O(1) size (technically $\Theta(\log n / \log \log n)$ w.h.p.)
- Also O(1) amortised insertion time
  - Insertion takes at least as much as an unsuccessful search, plus O(n) time when rehashing

Hash Table Practical Considerations

- Cache benefits
  - Looking up main memory several times adds a significant delay if the items are not cached. Using open addressing with a "hybrid" scheme avoids this delay.
    - Hybrid schemes for double-hash/quadratic probing and for cuckoo hash
    - Robin Hood hash is based on linear probing and avoids this delay.
    - In fact, closed addressing could also use chains of B-long vectors
- Parallelisation: Can check fingerprints of many slots in one step
- Adversarial inputs
  - Guarantees break down if the hash function used is not from a uniform hash family, and the inputs are crafted to exploit that
  - Can be avoided using (light-weight) cryptographically strong hash functions
  - Otherwise, in closed addressing, can fall back to BST for large buckets; in open addressing, migrate to BST/closed addressing table, or use a spillover stash